CHAPTER 12-1

순환 신경망 (RNN / LSTM)

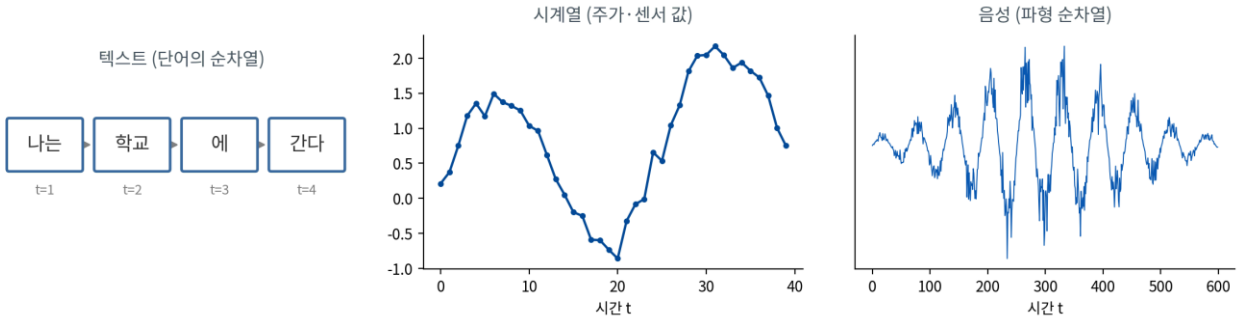
순서가 있는 데이터를 신경망은 어떻게 처리하는가?

학습 목표

- 순차 데이터의 특성과, 기존 신경망으로 순차 데이터를 다룰 때의 한계를 설명할 수 있습니다.
- 순환 신경망(RNN)의 구조와 순전파 연산을 수식으로 표현할 수 있습니다.
- BPTT 과정에서 기울기 소실이 발생하는 원인을 설명할 수 있습니다.
- LSTM 과 GRU 의 게이트 구조가 장기 의존성 문제를 어떻게 완화하는지 설명할 수 있습니다.
- Keras 로 RNN·LSTM 모형을 구현하고 성능을 비교할 수 있습니다.

순차 데이터

- 순차 데이터 (Sequential data)
  - 관측 값이 순서(order)를 가지고 나열되어 있는 데이터
    - t 번째 관측 값을 x(t) 로 표기하며, 전체를 x(1), x(2), … , x(T) 로 나타냄
    - 순서를 바꾸면 데이터의 의미 자체가 달라진다는 점이 가장 큰 특징
  - 자연어처리에서 다루는 대부분의 데이터가 순차 데이터에 해당



순차 데이터의 예시 (텍스트 · 시계열 · 음성)

순차 데이터

- 순차 데이터의 세 가지 특성
  - ① 순서 의존성 (order dependency)
    - "개가 사람을 물었다" 와 "사람이 개를 물었다" 는 단어 구성이 같지만 의미가 다름
  - ② 가변 길이 (variable length)
    - 어떤 문장은 3 단어, 다른 문장은 40 단어 → 입력의 크기를 고정할 수 없음
  - ③ 장기 의존성 (long-term dependency)
    - 현재 시점의 출력이 아주 먼 과거의 입력에 의해 결정되는 경우가 존재
      - 예 : "프랑스에서 자란 그는 … 유창한 (프랑스어) 를 말한다"

기존 신경망의 한계

- 다층 퍼셉트론(MLP)으로 순차 데이터를 처리하면?
  - 입력을 한 덩어리의 고정 길이 벡터로 밀어 넣어야 함 → 순서 정보가 사라짐
  - 문장 길이가 달라지면 입력 노드 수가 달라져 같은 모형을 사용할 수 없음
  - 시점마다 별도의 가중치를 두면 파라미터 수가 시점 수에 비례해 폭증

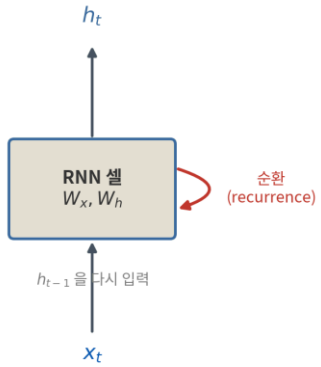


단어 순서를 무시하면 정반대의 의미를 구분할 수 없다

순환 신경망

- 순환 신경망 (RNN, Recurrent Neural Network)
  - 순차 데이터를 처리하기 위해 설계된 신경망 구조
    - 핵심 아이디어 : 이전 시점의 은닉 상태를 현재 시점의 입력과 함께 사용
  - 은닉 상태 (hidden state) h(t)
    - t 시점까지의 입력 x(1) … x(t) 를 요약해 담고 있는 고정 크기 벡터
    - 매 시점 갱신되어 다음 시점으로 전달됨 → "기억(memory)" 의 역할

기억을 가진 신경망
출력만 내보내는 MLP 와 달리
RNN 은 자기 자신에게도 값을
다시 흘려 보낸다.



RNN 의 순환 구조 (folded)

순환 신경망

- RNN 의 순전파 (forward propagation)
  - 은닉 상태 갱신식과 출력식
    - W_x : 입력 → 은닉 가중치, W_h : 은닉 → 은닉 가중치(순환 가중치)
    - b, c : 편향, 은닉층 활성함수로는 보통 tanh 를 사용

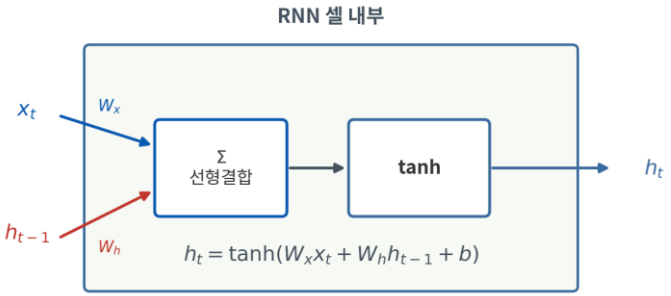
$$h(t) = \tanh(W_x \cdot x(t) + W_h \cdot h(t-1) + b)$$

$$y(t) = \text{softmax}(W_y \cdot h(t) + c)$$

초기 은닉 상태
h(0) 은 보통 영벡터로 초기화한다. 따라서 첫 시점의 은닉 상태는 x(1) 만으로 결정된다.

순환 신경망

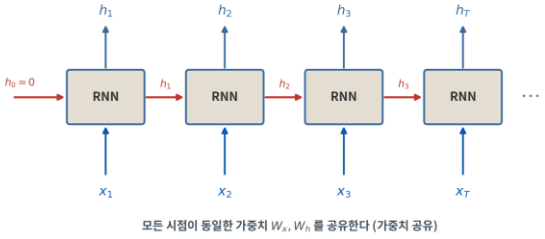
- RNN 셀 내부에서 일어나는 연산
  - 현재 입력 x(t) 와 직전 은닉 상태 h(t-1) 을 각각 선형 변환한 뒤 더한다
  - 더한 값에 tanh 를 적용해 새로운 은닉 상태 h(t) 를 얻는다
    - tanh 는 출력 범위를 (-1, 1) 로 제한해 은닉 상태가 발산하지 않도록 함



RNN 셀 내부의 연산 흐름

순환 신경망

- 펼친 구조 (unfolded / unrolled)
  - 순환 구조를 시간축으로 펼치면, 시점 수만큼 층이 쌓인 깊은 신경망과 같아진다
  - 가중치 공유 (weight sharing) : 모든 시점이 동일한 W_x, W_h 를 사용
    - 문장 길이가 달라져도 같은 파라미터로 처리 가능 → 가변 길이 입력 대응
    - 파라미터 수가 시점 수 T 와 무관하게 일정하게 유지됨



RNN 을 시간축으로 펼친 구조

순환 신경망 (예시)

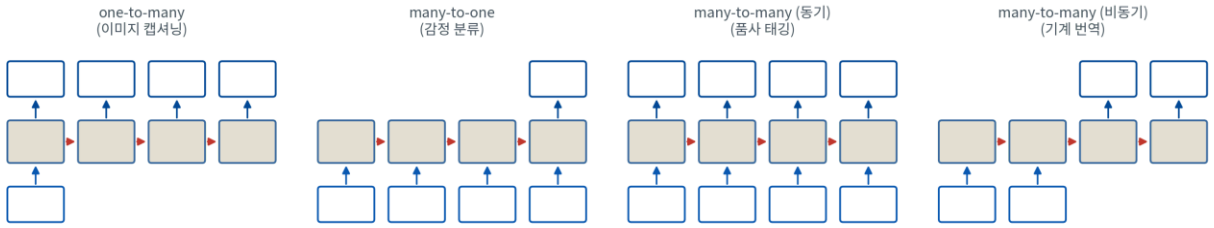
- 간단한 수치 예시 (은닉 차원이 1 인 RNN)
  - W_x = 0.5, W_h = 0.8, b = 0, h(0) = 0 이고 입력이 x = (1, 1, 0) 일 때

시점 t	입력 x(t)	W_x · x(t) + W_h · h(t -1)	h(t) = tanh(·)
1	1.0	0.5 × 1.0 + 0.8 × 0.00 0 = 0.500	0.462
2	1.0	0.5 × 1.0 + 0.8 × 0.46 2 = 0.870	0.701
3	0.0	0.5 × 0.0 + 0.8 × 0.70 1 = 0.561	0.509

관찰 포인트
· t = 3 에서 입력이 0 이지만 h(3) 은 0 이 아니다 → 과거 정보가 은닉 상태에 남아 있음
· 같은 입력 1.0 이 들어와도 h(1) 과 h(2) 의 값이 다르다 → 문맥에 따라 표현이 달라짐

순환 신경망

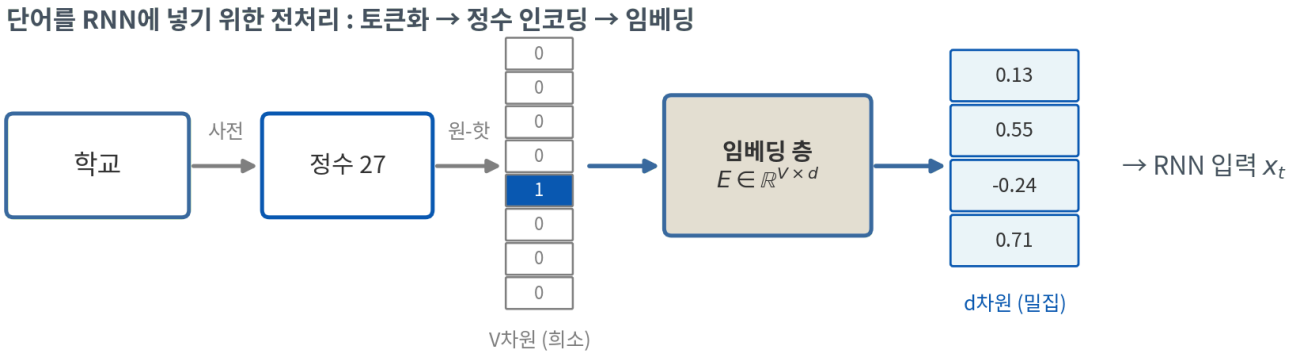
- 입력·출력 길이에 따른 RNN 구조의 분류
  - one-to-many : 하나의 입력 → 순차열 출력 (이미지 캡셔닝)
  - many-to-one : 순차열 입력 → 하나의 출력 (감정 분류, 스팸 판별)
  - many-to-many (동기) : 매 시점 입력과 출력이 대응 (품사 태깅, 개체명 인식)
  - many-to-many (비동기) : 입력을 모두 읽은 뒤 출력 생성 (기계 번역) → 12-2 절



RNN 의 네 가지 입출력 구조

RNN 의 입력 만들기

- 텍스트를 RNN 의 입력으로 바꾸는 과정
  - 토큰화 → 단어 사전 구축 → 정수 인코딩 → 임베딩
    - 원-핫 벡터는 차원이 어휘 크기 V 만큼 커지고 단어 간 유사도를 담지 못함
    - 임베딩 층은 V × d 크기의 행렬을 학습해 각 단어를 밀집 벡터로 변환 (d ≪ V)



정수 인코딩 → 임베딩 → RNN 입력

RNN 의 입력 만들기

- 가변 길이 문장의 처리 : 패딩(padding)과 마스킹(masking)
  - 미니배치 학습을 위해 한 배치 안의 문장 길이를 동일하게 맞춰야 함
    - 짧은 문장 뒤에 0(PAD) 토큰을 채워 최대 길이에 맞춤 (post-padding)
    - PAD 위치는 마스킹으로 연산에서 제외 → Keras 에서는 mask_zero=True 로 처리

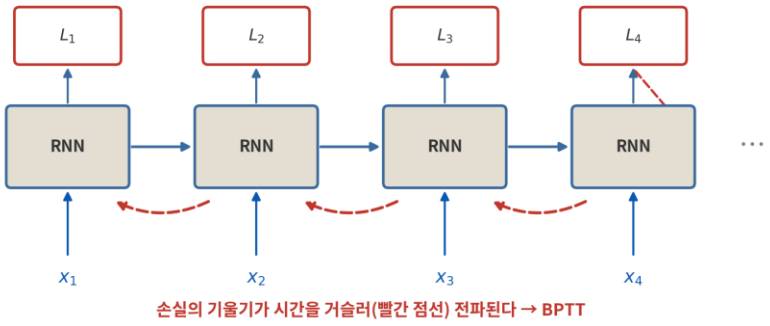


패딩으로 (batch, timesteps, dim) 형태를 맞춘다

RNN 의 학습

- 시간을 거슬러 가는 역전파 (BPTT, Backpropagation Through Time)
  - 펼친 구조에 통상적인 오차 역전파를 그대로 적용하는 방법
    - 전체 손실은 각 시점 손실의 합 : L = L(1) + L(2) + … + L(T)
    - W_h 의 기울기는 모든 시점의 기여를 더해서 계산됨

$$L = \sum_t L_t \,,\ \frac{\partial L}{\partial W_h} = \sum_t \frac{\partial L_t}{\partial W_h}$$



BPTT : 손실의 기울기가 시간 역방향으로 전파된다

RNN 의 학습

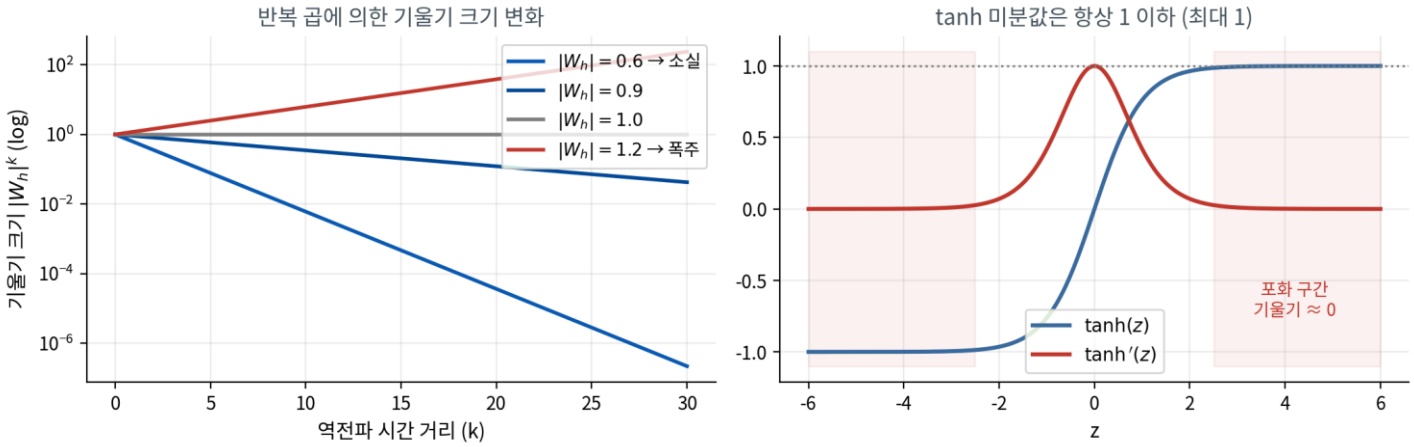
- BPTT 의 계산 비용과 잘라낸 BPTT
  - 문장이 길어지면 펼친 층수가 늘어나 메모리·연산량이 함께 증가
    - 역전파를 위해 시점 수 T 에 비례하는 만큼의 순전파 결과를 저장해야 함
  - 잘라낸 BPTT (Truncated BPTT)
    - 역전파 구간을 일정 길이 k 로 잘라서 수행 (순전파는 계속 이어짐)
    - 장점 : 메모리 절약, 학습 속도 향상
    - 단점 : k 보다 먼 과거의 의존 관계는 학습되지 않음

실무 관점
대부분의 프레임워크는 입력 순차열 길이만큼만 펼쳐서 학습한다.
따라서 maxlen (자를 문장 길이) 이 곧 학습 가능한 의존 거리의 상한이 된다.

기울기 소실 문제

- 기울기 소실(vanishing) 과 기울기 폭주(exploding)
  - k 시점 이전으로 기울기를 전파하려면 야코비 행렬을 k 번 곱해야 한다

$\partial h(t) / \partial h(t-k) = \Pi\ (W_h)^T \cdot \mathrm{diag}(\tanh'(\cdot)) \Rightarrow$ 대략 $|W_h|^k$ 에 비례



반복 곱에 의한 기울기 크기 변화와 tanh 미분값

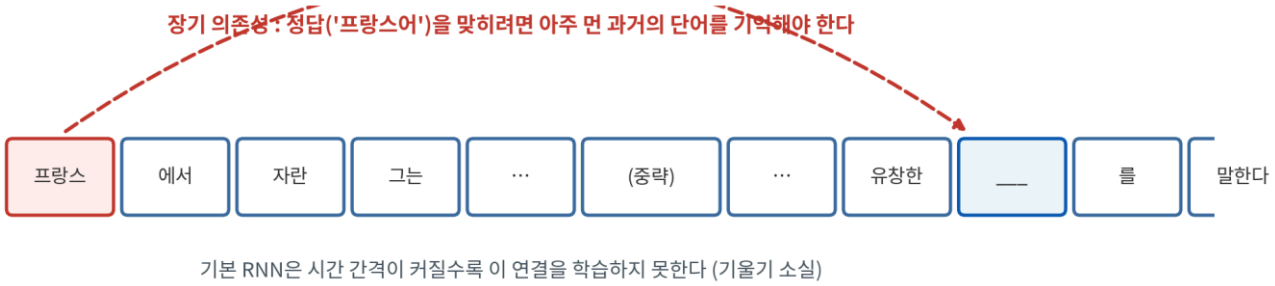
기울기 소실 문제

- 왜 반복 곱이 문제가 되는가
  - | W_h | < 1 인 경우 → 기울기가 지수적으로 0 에 수렴 (기울기 소실)
    - 먼 과거에 대한 가중치가 거의 갱신되지 않아 장기 의존성을 학습하지 못함
  - | W_h | > 1 인 경우 → 기울기가 지수적으로 발산 (기울기 폭주)
    - 손실이 NaN 이 되거나 학습이 발산함
  - tanh 의 미분값은 최대 1 이고, 포화 구간에서는 거의 0
    - 곱해질수록 기울기가 더욱 작아지는 방향으로 작용

문제	증상	대표적 해결 방법
기울기 폭주	손실 발산 / NaN	기울기 클리핑 (gradient clipping)
기울기 소실	먼 과거를 학습하지 못함	게이트 구조 (LSTM, GRU)

장기 의존성 문제

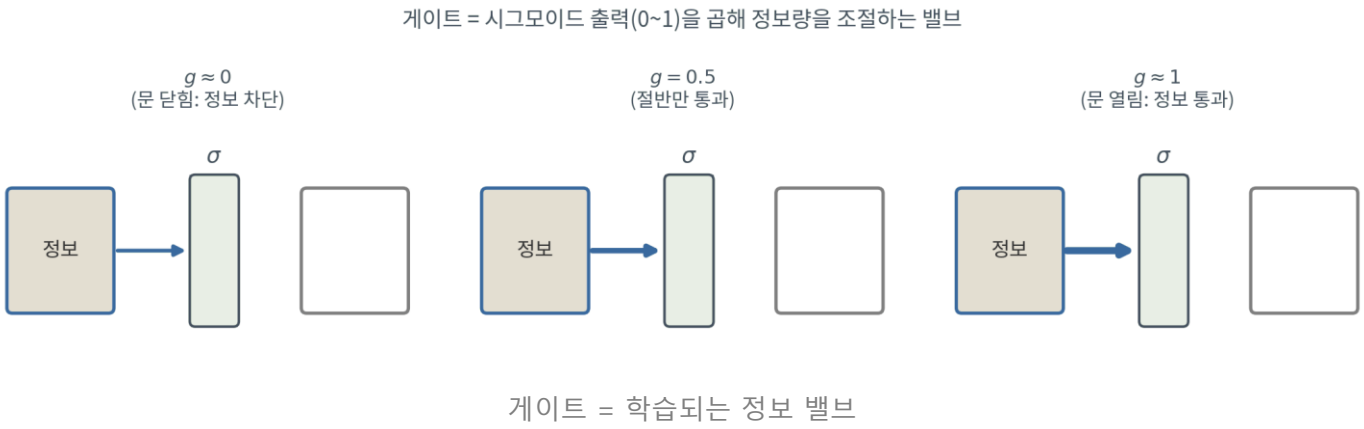
- 장기 의존성 (long-term dependency)
  - 정답을 맞히기 위해 필요한 정보가 아주 먼 과거의 입력에 담겨 있는 상황
  - 기본 RNN 은 은닉 상태가 매 시점 tanh 로 덮어써지면서 과거 정보가 희석됨
    - 실험적으로 대략 10 시점 이상 (Bengio et al., 1994) 떨어진 의존 관계는 학습이 매우 어렵다



먼 과거의 단어가 현재의 예측을 결정하는 예

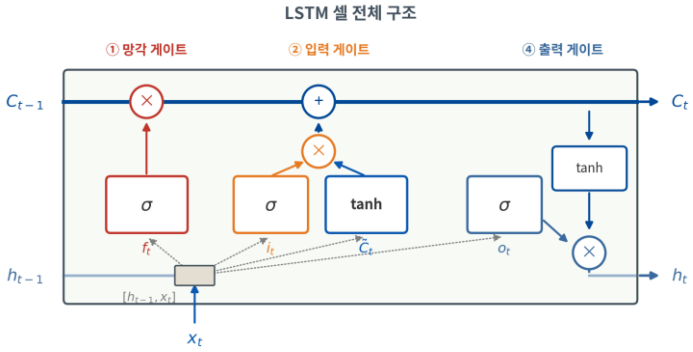
게이트 구조의 아이디어

- 게이트 (gate)
  - 정보를 "얼마나 통과시킬지" 를 학습하는 밸브
    - 시그모이드 출력 $g \in (0, 1)$ 을 정보 벡터에 원소별로 곱해 통과량을 조절
    - $g \approx 0$ 이면 차단, $g \approx 1$ 이면 그대로 통과
  - 게이트의 값 자체도 학습되는 가중치로 결정된다는 점이 핵심



LSTM

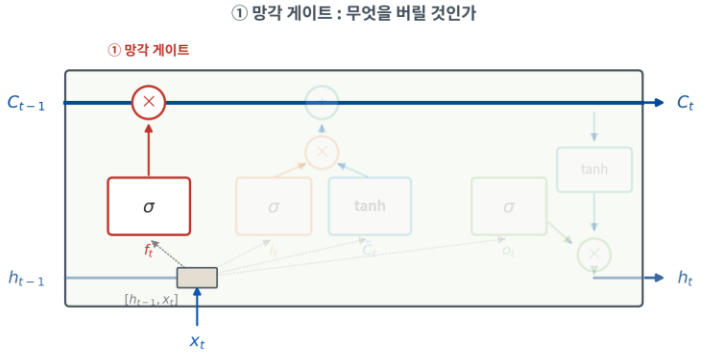
- LSTM (Long Short-Term Memory)
  - Hochreiter & Schmidhuber (1997) 가 제안한 게이트 기반 순환 구조
  - (망각 게이트는 Gers et al. (2000) 이 추가하여 오늘날의 3-게이트 표준 구조가 됨)
  - 두 개의 상태를 유지
    - 셀 상태 C(t) : 장기 기억 — 컨베이어 벨트처럼 곧게 흐름
    - 은닉 상태 h(t) : 단기 기억 및 현재 시점의 출력
  - 세 개의 게이트 : 망각 f(t) · 입력 i(t) · 출력 o(t)



LSTM 셀의 전체 구조

LSTM (① 망각 게이트)

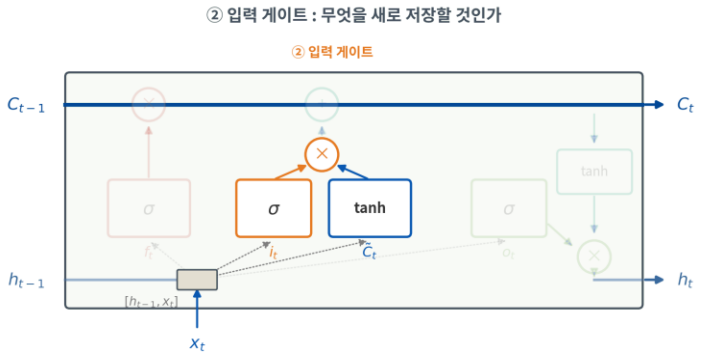
- ① 망각 게이트 (forget gate)
  - 직전 셀 상태 C(t-1) 에서 무엇을 버릴지 결정
    - $f(t) = \sigma (W_f \cdot [h(t-1), x(t)] + b_f), \quad f(t) \in (0, 1)$
    - $f(t) \odot C(t-1)$: 원소별로 곱해 필요 없는 기억을 지운다
      - 예 : 새로운 주어가 등장하면 이전 주어의 성·수 정보를 잊는다



① 망각 게이트 : 무엇을 버릴 것인가

LSTM (② 입력 게이트)

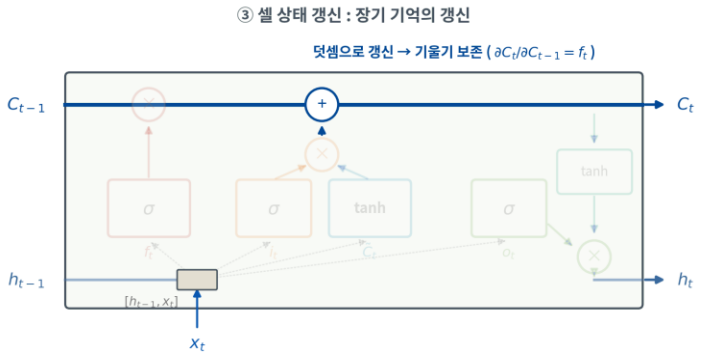
- ② 입력 게이트 (input gate) 와 후보값
  - 현재 입력에서 무엇을 새로 저장할지 결정
    - $i(t) = \sigma (W_i \cdot [h(t-1), x(t)] + b_i)$: 저장 비율
    - $\tilde{C}(t) = \tanh (W_C \cdot [h(t-1), x(t)] + b_C)$: 저장할 후보값
    - $i(t) \odot \tilde{C}(t)$: 실제로 셀 상태에 더해질 새 정보



② 입력 게이트 : 무엇을 새로 저장할 것인가

LSTM (③ 셀 상태 갱신)

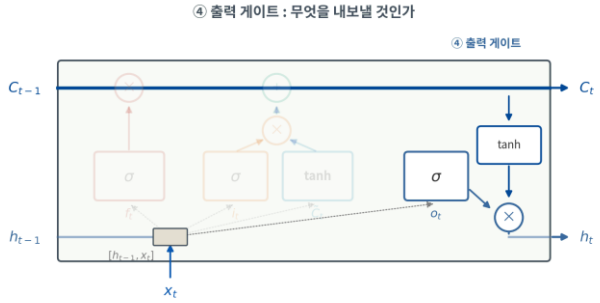
- ③ 셀 상태 갱신 (cell state update)
  - C(t) = f(t) ⊙ C(t-1) + i(t) ⊙ C(t̃)
    - 앞의 항은 "남길 과거 기억", 뒤의 항은 "추가할 새 기억"
    - 곱이 아니라 덧셈으로 갱신된다는 점이 기울기 소실 완화의 핵심
      - ∂C(t) / ∂C(t-1) ≈ f(t) (게이트를 상수로 본 직접 경로 기준) 이므로 f(t) ≈ 1 이면 기울기가 그대로 보존된다



③ 셀 상태 갱신 : 곱이 아닌 덧셈으로 갱신

LSTM (④ 출력 게이트)

- ④ 출력 게이트 (output gate)
  - 갱신된 셀 상태 중 무엇을 이번 시점의 출력으로 내보낼지 결정
    - $o(t) = \sigma (W_o \cdot [h(t-1), x(t)] + b_o)$
    - $h(t) = o(t) \odot \tanh (C(t))$
      - 장기 기억 C(t) 는 그대로 유지하면서, 필요한 부분만 골라 출력한다



④ 출력 게이트 : 무엇을 내보낼 것인가

LSTM

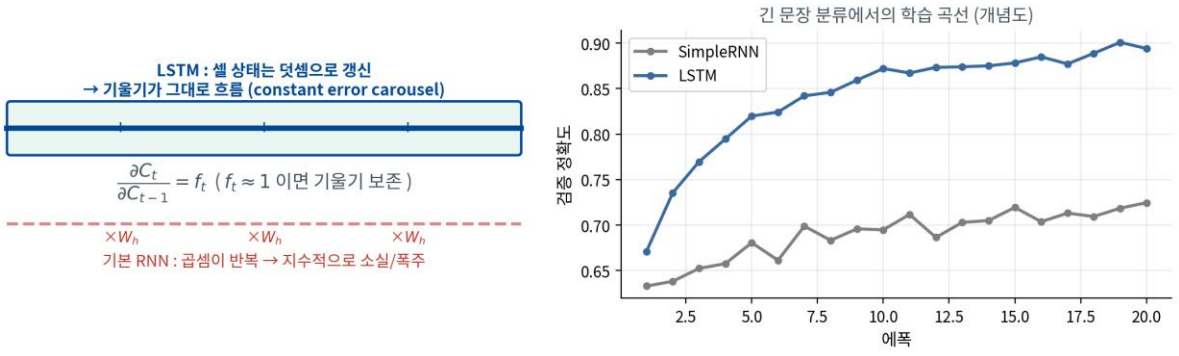
◦ LSTM 수식 정리

구성 요소	수식	역할
망각 게이트	$f(t) = \sigma (W_f [h(t-1), x(t)] + b_f)$	과거 기억 삭제 비율
입력 게이트	$i(t) = \sigma (W_i [h(t-1), x(t)] + b_i)$	새 정보 저장 비율
후보값	$\tilde{C}(t) = \tanh (W_C [h(t-1), x(t)] + b_C)$	저장할 새 정보
셀 상태	$C(t) = f(t) \odot C(t-1) + i(t) \odot \tilde{C}(t)$	장기 기억 갱신
출력 게이트	$o(t) = \sigma (W_o [h(t-1), x(t)] + b_o)$	출력 비율
은닉 상태	$h(t) = o(t) \odot \tanh (C(t))$	단기 기억 · 출력

파라미터 수
게이트 3 개 + 후보값 1 개 = 4 개의 선형 변환이 필요하므로,
LSTM 의 파라미터 수는 같은 은닉 차원의 기본 RNN 대비 약 4 배가 된다.

LSTM

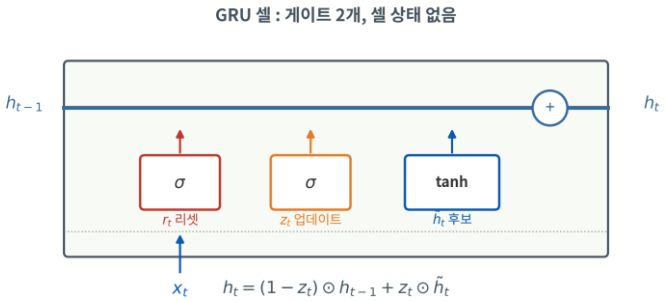
- LSTM 이 기울기 소실을 완화하는 이유
  - 셀 상태는 게이트를 통한 덧셈으로 갱신되므로 기울기가 곧게 흐른다
    - ∂C(t) / ∂C(t-1) = f(t) : 반복 곱해지는 항이 학습되는 값 f(t) 로 대체됨
    - 망각 게이트가 열려 (f(t) ≈ 1) 있으면 먼 과거까지 기울기가 전달됨



기울기 흐름 비교와 학습 곡선 (개념도)

GRU

- GRU (Gated Recurrent Unit)
  - Cho et al. (2014) 이 제안한 LSTM 의 경량화 구조 (아래 식은 Chung et al. 2014 표기이며 Keras·PyTorch 구현은 z 와 1-z 의 위치가 반대)
    - 셀 상태를 없애고 은닉 상태 h(t) 하나만 유지
    - 게이트를 2 개로 축소 : 리셋 게이트 r(t), 업데이트 게이트 z(t)
    - h(t) = (1 - z(t)) ⊙ h(t-1) + z(t) ⊙ h̃(t) — 삭제와 추가를 한 게이트가 담당



GRU 셀의 구조

LSTM 과 GRU 비교

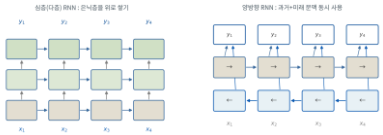
◦ 어떤 구조를 선택할 것인가

항목	SimpleRNN	LSTM	GRU
유지하는 상태	h	h , C	h
게이트 수	0	3	2
파라미터 (상대)	1 배	약 4 배	약 3 배
장기 의존성	약함	강함	강함
학습 속도	빠름	느림	중간
권장 상황	짧은 순차열 · 교육용	긴 순차열 · 성능 우선	데이터·자원이 적을 때

실험적 결론
· 데이터셋에 따라 LSTM 과 GRU 의 우열이 갈리므로 두 구조를 모두 시도하는 것이 일반적이다.
· 다만 두 구조 모두 SimpleRNN 보다는 대체로 우수한 성능을 보인다.

RNN 의 확장

- 구조를 더 깊게, 더 넓게
  - 심층(다층) RNN : 은닉층을 위로 쌓아 표현력을 높임
    - Keras 에서는 아래쪽 층에 return_sequences=True 를 지정해 연결
  - 양방향(bidirectional) RNN : 정방향·역방향 RNN 을 함께 사용
    - 현재 시점에서 과거와 미래 문맥을 모두 참조 → 품사 태깅·개체명 인식에 유리
      - 단, 미래 입력을 알 수 없는 실시간 생성 과제에는 사용할 수 없음



심층 RNN 과 양방향 RNN

Keras 구현

◦ Keras 로 작성한 RNN / LSTM 분류 모형

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, LSTM, Dense

model = Sequential([
    Embedding(input_dim=10001, output_dim=64, mask_zero=True)   # 0 은 PAD 전용이므로 input_dim = 어휘수 + 1,
    LSTM(64),                          # SimpleRNN(64) 으로 바꿔 비교
    Dense(1, activation='sigmoid'),  # 이진 감정 분류
])
model.compile(optimizer='adam', loss='binary_crossentropy',
              metrics=['accuracy'])
```

실습에서 확인할 점

· SimpleRNN 과 LSTM 의 검증 정확도 차이

· 문장 최대 길이(maxlen)를 늘렸을 때 두 모형의 성능 변화

제출물 : 실습 과제를 수행하고 실행 결과가 포함된 .ipynb 파일을 제출

정리

- 이번 절의 요약
  - 순차 데이터는 순서 의존성 · 가변 길이 · 장기 의존성이라는 특성을 가진다
  - RNN 은 은닉 상태를 순환시켜 과거를 요약하며, 모든 시점이 가중치를 공유한다
  - BPTT 과정에서 야코비 행렬이 반복 곱해져 기울기 소실 · 폭주가 발생한다
  - LSTM 은 셀 상태와 3 개의 게이트로, GRU 는 2 개의 게이트로 이 문제를 완화한다
  - 다음 절에서는 입력과 출력의 길이가 다른 문제(기계 번역)를 다루기 위한
    - 순차열 대 순차열(Seq2Seq) 모형과 주의 메커니즘을 학습한다

질문 및 정리

- 생각해 볼 문제
  - ① RNN 에서 가중치를 시점마다 다르게 두면 어떤 문제가 생길까?
  - ② 기울기 클리핑은 소실과 폭주 중 어느 문제를 해결하는가? 그 이유는?
  - ③ 망각 게이트의 편향 b_f 를 크게 초기화하면 학습 초기에 어떤 효과가 있을까?
  - ④ 양방향 LSTM 을 실시간 음성 인식에 사용할 수 없는 이유는 무엇인가?

과제
실습 1 (RNN / LSTM) 노트북을 수행하고 (실습 2·3은 12-2 에서 부과)
각 노트북 마지막의 [과제] 셀을 완성하여 제출한다.